PHP GET vs POST Methods

PHP GET and POST methods are the two most commonly used **HTTP request methods** to send form data from a client (usually a browser) to a server.

They determine **how** the data is sent and how it is **accessed** in PHP using superglobal arrays:

- `$_GET[]`
- `$_POST[]`

1. GET Method

GET sends data as **URL parameters**. The data is visible in the browser's address bar.

```
// Accessed via URL: example.com/page.php?name=John
echo $_GET['name']; // Outputs: John
```

Characteristics:

- Data is appended to the URL: `example.com/page.php?name=John&age=30`
- Uses `$_GET` superglobal array in PHP.
- Bookmarked easily (URL remains the same).
- Cached by browsers.
- Limited data transfer (usually 2048 characters).

Use Cases:

- Search engines (`?q=term`)
- Filter options
- Pagination (`?page=2`)
- Sorting (`?sort=asc`)

Security Concerns:

- Data is visible in the address bar.
- Never use for sensitive information (e.g., passwords).
- Easily tampered with (use validation/sanitization).

Example:

```
<form method="GET" action="search.php">
  <input type="text" name="query" placeholder="Search...">
  <button type="submit">Search</button>
</form>
// search.php
$query = htmlspecialchars($_GET['query']);
echo "You searched for: $query";
```

2. POST Method

POST sends form data in the **HTTP request body**. The data is not visible in the URL.

```
echo $_POST['email']; // Outputs submitted email
```

Characteristics:

- Data is sent in the HTTP body, not visible in the URL.
- Uses `$_POST` superglobal array.
- Cannot be bookmarked or cached.
- No size limit (technically — set by server config).
- Supports uploading files via `<input type="file">`.

Use Cases:

- Login forms
- Registration forms
- Updating user information
- File uploads

Security Tips:

- Use HTTPS to encrypt data in transit.
- Sanitize and validate all inputs.
- Combine with tokens for CSRF protection.

Example:

```
<form method="POST" action="submit.php">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="submit">Login</button>
</form>
// submit.php
$username = trim($_POST['username']);
$password = $_POST['password'];
```

GET vs POST Comparison Table

Feature	GET	POST
Visibility	Data is visible in the URL	Data is hidden (in request body)
Data Length	Limited (~2000 chars)	Virtually unlimited
Bookmarkable	Yes	No
Cacheable	Yes	No
Security	Less secure	More secure with HTTPS
Use Case	Search, filters	Login, registration, uploads

PHP Sessions

What is a Session?

- A **session** is a way to store data across multiple requests.
- The data is stored on the **server**.
- The browser receives a **Session ID** in a cookie (`PHPSESSID`).

How It Works:

1. `session_start()` starts or resumes a session.
2. PHP assigns a unique ID (e.g., `a9sd8f34r...`).
3. This ID is stored as a cookie in the browser.
4. PHP uses this ID to fetch session data from the server.

Creating a Session:

```
session_start();
$_SESSION["username"] = "JohnDoe";
```

Accessing Session Data:

```
session_start();
echo $_SESSION["username"];
```

Destroying a Session:

```
session_start();
session_unset();    // Unset all session variables
session_destroy();  // End session
```

Use Cases:

- User login/logout
- E-commerce shopping cart
- Role-based access control
- Multi-step forms

Tips for Secure Sessions:

- Always use `session_start()` at the top.
- Use `session_regenerate_id()` on login to prevent fixation attacks.
- Set `session.cookie_httponly` and `session.cookie_secure` in `php.ini`.

PHP Cookies

What is a Cookie?

- A cookie is a **key-value pair stored in the browser**.

- Sent with every HTTP request to the server.
- Persistent — remains after the session ends.

Syntax:

```
setcookie(name, value, expire, path, domain, secure, httponly);
```

Setting a Cookie:

```
setcookie("user", "John", time() + (86400 * 7)); // Valid for 7 days
```

Reading a Cookie:

```
echo $_COOKIE["user"];
```

Deleting a Cookie:

```
setcookie("user", "", time() - 3600); // Set expiration in the past
```

Use Cases:

- Remembering user preferences
- “Remember Me” on login
- Tracking user behavior
- A/B testing

Cookie Security:

- Use `secure` flag for HTTPS only cookies.
- Use `httponly` to prevent JavaScript access (protection from XSS).
- Encrypt sensitive values before storing.

Summary: When to Use What?

Feature	GET	POST	SESSION	COOKIE
Stored In	URL	Request Body	Server	Client (Browser)
Visibility	Visible in address bar	Hidden from user	Hidden from user	Visible via dev tools
Capacity	Limited	Unlimited (practical)	Unlimited (server-side)	~4KB max
Persistence	One request	One request	Until browser/session ends	Persistent (set manually)
Security	Least secure	More secure with HTTPS	Most secure (server-side)	Least secure unless encrypted